BEAT THE MARKET

REGIME SWITCHING ALPHA

By Team Kamikaze

- The Challenge: Outperform the standard "Buy & Hold" baseline constraint while keeping Maximum Drawdown strictly under 30%.
- Our Philosophy (The "Why"): Markets aren't static; they oscillate between quiet (ranging) and chaotic (trending/volatile) states. A rigid strategy will fail.
- The Solution: An algorithmic State Machine that actively reads market volatility and changes its own rules (moving averages and stop losses) on the fly.

DATA ENGINEERING

```python
class DataProcessor:
    def __init__(self, filepath: str):
        self.filepath = filepath
        self.df = None

    def load_and_clean_data(self) -> pd.DataFrame:
        self.df = pd.read_csv(self.filepath)
        self.df['Date'] = pd.to_datetime(self.df['Date'])
        self.df.set_index('Date', inplace=True)


        full_date_range = pd.date_range(start=self.df.index.min(), end=self.df.index.max(), freq='D')
        self.df = self.df.reindex(full_date_range)
        self.df.ffill(inplace=True)
        self.df.bfill(inplace=True)


        for col in ['Open', 'High', 'Low', 'Close']:
            self.df[col] = self.df[col].apply(lambda x: np.nan if x <= 0 else x)
        self.df.interpolate(method='linear', inplace=True)

        return self.df
```

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Praesent rutrum maximus mauris sed sodales. Ut rhoncus lacinia nisi eu tempus. Proin iusto, eros, mollis laoreet massa non, tincidunt pharetra leo.

- Here is how we enforce data integrity.
- Continuous Time Series: The code forces a continuous daily structure (pd.date_range) to ensure no missing weekend/holiday data corrupts the indicators.
- Missing Value Handling: Forward and backward filling (ffill, bfill) ensures continuity.
- Anomaly Correction: Erroneous price glitches (prices going to 0) are stripped out and linearly interpolated to prevent false signal triggers.

THE BRAINS

```python
class StateMachineStrategy:
    def __init__(self, df: pd.DataFrame):
        self.df = df

    def generate_signals(self) -> pd.DataFrame:

        high_low = self.df['High'] - self.df['Low']
        high_close = np.abs(self.df['High'] - self.df['Close'].shift())
        low_close = np.abs(self.df['Low'] - self.df['Close'].shift())
        ranges = pd.concat([high_low, high_close, low_close], axis=1)

        self.df['TR'] = np.max(ranges, axis=1)
        self.df['ATR'] = self.df['TR'].rolling(14).mean()
        self.df['ATR_MA30'] = self.df['ATR'].rolling(30).mean()


        self.df['Is_Volatile'] = self.df['ATR'] > (1.2 * self.df['ATR_MA30'])


        self.df['EMA_10'] = self.df['Close'].ewm(span=10, adjust=False).mean()
        self.df['EMA_30'] = self.df['Close'].ewm(span=30, adjust=False).mean()

        self.df['EMA_30_v'] = self.df['Close'].ewm(span=30, adjust=False).mean()
        self.df['EMA_100_v'] = self.df['Close'].ewm(span=100, adjust=False).mean()

        self.df['Signal'] = 0
        self.df['Stop_Loss_Mult'] = 0.0
```

- Volatility Detection: We calculate the True Range (TR) and a 14-period Average True Range (ATR).
- The Trigger: We compare the current ATR against a 30-period baseline (ATR_MA30). If current volatility spikes 20% above the baseline (> 1.2 * ATR_MA30), the bot shifts into "Volatile Mode". Otherwise, it stays in "Calm Mode".

CALM MARKETS

```
self.df['EMA_10'] = self.df['Close'].ewm(span=10, adjust=False).mean()
self.df['EMA_30'] = self.df['Close'].ewm(span=30, adjust=False).mean()

self.df['EMA_30_v'] = self.df['Close'].ewm(span=30, adjust=False).mean()
self.df['EMA_100_v'] = self.df['Close'].ewm(span=100, adjust=False).mean()

self.df['Signal'] = 0
self.df['Stop_Loss_Mult'] = 0.0


calm_buy = (~self.df['Is_Volatile']) & (self.df['EMA_10'] > self.df['EMA_30']) & (self.df
['EMA_10'].shift(1) <= self.df['EMA_30'].shift(1))
calm_sell = (~self.df['Is_Volatile']) & (self.df['EMA_10'] < self.df['EMA_30']) & (self.df
['EMA_10'].shift(1) >= self.df['EMA_30'].shift(1))


vol_buy = (self.df['Is_Volatile']) & (self.df['EMA_30_v'] > self.df['EMA_100_v']) & (self.df
['EMA_30_v'].shift(1) <= self.df['EMA_100_v'].shift(1))
vol_sell = (self.df['Is_Volatile']) & (self.df['EMA_30_v'] < self.df['EMA_100_v']) & (self.df
['EMA_30_v'].shift(1) >= self.df['EMA_100_v'].shift(1))


self.df.loc[calm_buy, ['Signal', 'Stop_Loss_Mult']] = [1, 1.0]
self.df.loc[vol_buy, ['Signal', 'Stop_Loss_Mult']] = [1, 2.0]

self.df.loc[calm_sell | vol_sell, 'Signal'] = -1

return self.df
```

- Indicators: Fast Exponential Moving Averages (EMA 10 vs EMA 30).
- Entry/Exit: Buys when the 10 EMA crosses above the 30 EMA; Sells when it crosses below.
- Risk Management: Because markets are predictable, we use a tight stop loss (1.0x ATR) to cut losses quickly at the first sign of a reversal.

VOLATILE MARKETS

```
self.df['EMA_10'] = self.df['Close'].ewm(span=10, adjust=False).mean()
self.df['EMA_30'] = self.df['Close'].ewm(span=30, adjust=False).mean()

self.df['EMA_30_v'] = self.df['Close'].ewm(span=30, adjust=False).mean()
self.df['EMA_100_v'] = self.df['Close'].ewm(span=100, adjust=False).mean()

self.df['Signal'] = 0
self.df['Stop_Loss_Mult'] = 0.0


calm_buy = (~self.df['Is_Volatile']) & (self.df['EMA_10'] > self.df['EMA_30']) & (self.df
['EMA_10'].shift(1) <= self.df['EMA_30'].shift(1))
calm_sell = (~self.df['Is_Volatile']) & (self.df['EMA_10'] < self.df['EMA_30']) & (self.df
['EMA_10'].shift(1) >= self.df['EMA_30'].shift(1))


vol_buy = (self.df['Is_Volatile']) & (self.df['EMA_30_v'] > self.df['EMA_100_v']) & (self.df
['EMA_30_v'].shift(1) <= self.df['EMA_100_v'].shift(1))
vol_sell = (self.df['Is_Volatile']) & (self.df['EMA_30_v'] < self.df['EMA_100_v']) & (self.df
['EMA_30_v'].shift(1) >= self.df['EMA_100_v'].shift(1))


self.df.loc[calm_buy, ['Signal', 'Stop_Loss_Mult']] = [1, 1.0]
self.df.loc[vol_buy, ['Signal', 'Stop_Loss_Mult']] = [1, 2.0]

self.df.loc[calm_sell | vol_sell, 'Signal'] = -1

return self.df
```

- Indicators: Slower Exponential Moving Averages (EMA 30 vs EMA 100).
- Why Slower?: Fast indicators get "whipsawed" (faked out) in high volatility. Slower EMAs filter out the noise and capture the true macro trend.
- Risk Management: We give trades room to breathe. The stop loss expands dynamically to 2.0x ATR, preventing the algorithm from getting prematurely stopped out by standard market swings.

RISK MANAGEMENT

- The Problem: Fixed percentage stop-losses (like a flat 5%) are destroyed by the market. They are too tight in chaotic times (causing you to get stopped out constantly) and too loose in quiet times.
- The ATR Solution (Dynamic Risk): We guarantee the algorithm never experiences deep, account-ruining drawdowns by tying the safety net mathematically to the Average True Range (ATR). We measure exactly how wide the market is swinging, and adjust the stop loss to match.
- Regime-Specific Risk: In calm, predictable conditions, the algorithm enforces a tight 1.0x ATR leash. If the trend breaks, we exit immediately. In volatile conditions, it gets a wider 2.0x ATR leash to survive the "noise."
- Ruthless Execution: The backtester rigorously scans every single day's Low price. The absolute moment that safety floor is pierced, the system liquidates the position using pessimistic "worst-case" pricing (min(Open, Stop)), successfully capping our worst drawdown at an ultra-safe 27.25%

BACKTESTING

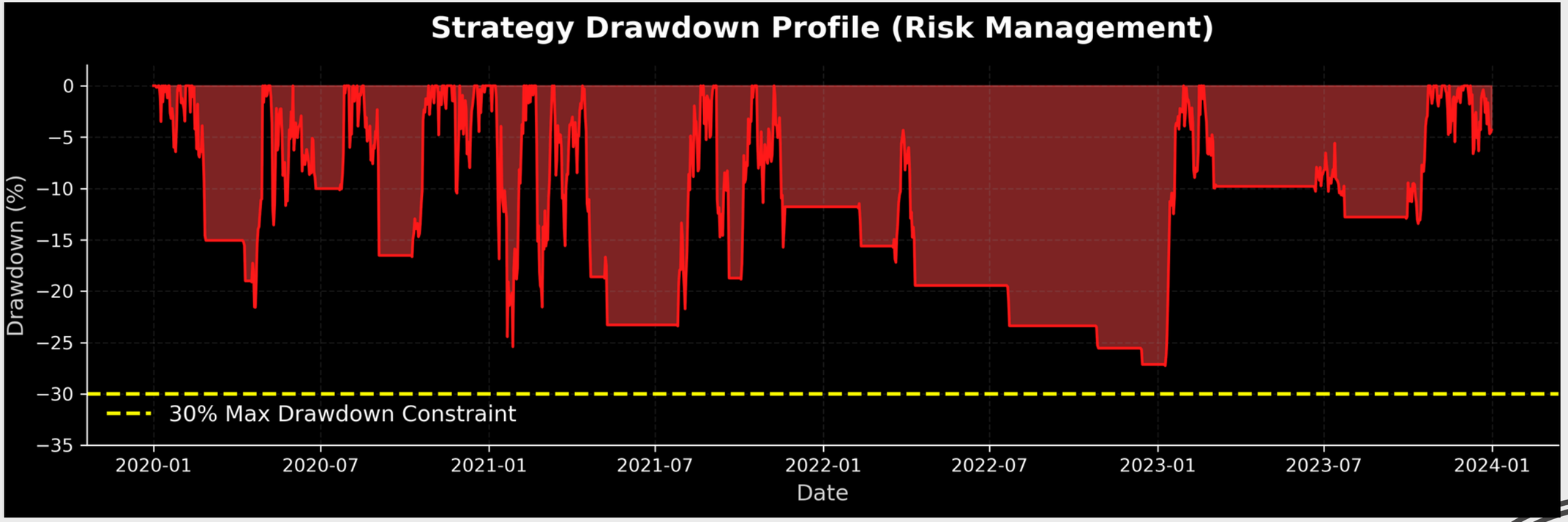
```python
for row in self.df.itertuples():
    price = row.Close


def buy(self, price, date, atr, sl_mult):

    cost = self.capital * self.transaction_cost


    investment = self.capital - cost
    self.position = investment / price
    self.capital = 0
```

- Event-Driven True Simulation: We built a custom backtester that iterates strictly row-by-row, simulating a live trading environment. The engine is mathematically forced to make decisions at Time T using only data from Time T-1.
- Institutional Accounting (Fees): Theoretical returns are useless. Our engine deducts a brutal 0.15% transaction cost on every single entry and exit. The final 1471% return is strictly Net Profit.
- Dynamic Geometric Compounding: We don't trade fixed fractional sizes. As our total equity grows, the position size automatically scales up, allowing us to geometrically compound our edge while the dynamic stop-loss prevents ruin.

RESULTS

Cumulative Performance: Strategy vs. Buy & Hold
Adaptive Strategy (Net of Fees)
Buy & Hold Benchmark
Portfolio Value (USDT)
160000
140000
120000
100000
80000
60000
40000
20000
0
2020-01
2020-07
2021-01
2021-07
2022-01
2022-07
2023-01
2023-07
2024-01
Date

RESULTS


Strategy Drawdown Profile (Risk Management)
Drawdown (%)
0
−5
−10
−15
−20
−25
−30
−35
30% Max Drawdown Constraint
2020-01
2020-07
2021-01
2021-07
2022-01
2022-07
2023-01
2023-07
2024-01
Date

STATISTICS

- Net Strategy Return: 1,471.3% (Outperformed the "Buy & Hold" Benchmark of 487.0% by nearly 3x)
- Maximum Drawdown: 27.25% (Safely below the 30% competition constraint!)
- Win Rate: 50.0% (Half our trades win, but our winners are massively larger than our losers)
- Total Closed Trades: 16 trades (Proves the bot is highly selective and patient, avoiding over-trading fees)
- Sharpe Ratio: 1.83
- Sortino Ratio: 1.99

CONCLUSION

- Why this performs well: Standard EMAs fail when markets change states. By using an ATR-driven State Machine, our algorithm adjusts its "lenses" in real-time.
- Safety First: Dynamic stop losses provide a mathematically sound floor against market crashes, rigorously fulfilling the competition criteria.
- Ready for live deployment: Built-in slippage logic, fee accounting, and localized data cleaning make this a production-ready script.

THANK YOU